

Universidad Nacional Del Oeste

Escuela de Ciencias Tecnológicas

Programación literaria verificada formalmente

Autor: Ian Alén Cisneros, DNI: 46193882

Tutor: Lic. Laura Mercedes Solla

Trabajo Final presentado para obtener el título de

Licenciado de Informática

October - 2026

Resumen

El uso de modelos de lenguaje está cada vez más extendido. Se ha documentado su capacidad de acelerar el ciclo de desarrollo de software (Kumar et al., 2025), aunque presenta límites derivados de su naturaleza probabilística (Mohsin et al., 2024). La mejora en su nivel de determinismo y verificación en el software creado, mejoraría de manera directa la calidad y velocidad del desarrollo de software. En los últimos años, las empresas e instituciones están delegando cada vez más responsabilidades dentro del ciclo de vida del desarrollo de software a los modelos de inteligencia artificial. En particular, su eficacia en la generación de código resulta útil en la construcción del software, aumentando significativamente la velocidad del ciclo de desarrollo. Sin embargo, aparecen limitaciones que provienen de la naturaleza estocástica de estos modelos probabilísticos (alucinaciones) y la limitación temporal de la auditoría de código por parte de los humanos, lo que ralentiza su uso en sistemas de alta criticidad ágil y fiablemente. A través de los años, se han ido desarrollando diferentes metodologías, técnicas y herramientas para disminuir los errores en el ciclo de vida tradicional (Sorokin et al., 2024). Creemos que es posible integrar estas facilidades con los modelos de lenguajes para alcanzar la mayor fiabilidad posible en la construcción del software, en particular para los programas protocolares de salud, mediante un enfoque neuro-simbólico que combine verificación formal y programación literaria (Ma et al., 2025; Zhang et al., 2024).

Palabras clave: modelos de lenguaje avanzado, programación literaria, verificación formal, triage de Manchester

Índice

Capítulo 1 — Introducción

Planteamiento del problema 1

Falta de determinismo y verificabilidad 1

Auditoría humana insostenible 1

Falta de integración sistemática 1
Preguntas e hipótesis 2
Pregunta principal 2
Preguntas secundarias 2
Hipótesis 2
Objetivos 2
Objetivo general 2
Objetivos específicos 3
Justificación 3
Marco de referencia 4
Programación literaria 4
Verificación formal y asistentes de pruebas 4
Sistemas de triage y software clínico 4
Estado del arte 4
Aspectos metodológicos 5
Paradigma 5
Método 5
Técnicas cuantitativas 5
Técnicas cualitativas 6
Capítulo 2 — Marco teórico
LLMs y generación de código 7
Verificación formal y Lean 4 7
Programación literaria 7
Auditoría humana y trazabilidad 7
Capítulo 3 — Estado del arte

Generación de código asistida por LLMs 8

Traducción de requisitos a especificaciones formales 8

Programación literaria y auditoría en la era de los LLMs 8

Capítulo 4 — Marco metodológico

Paradigma y enfoque 9

Diseño de la investigación 9

Técnicas de recolección y análisis 9

Protocolo experimental 9

Capítulo 5 — Diseño e implementación del prototipo

Arquitectura del sistema 10

Generación de código a partir de requisitos clínicos 10

Verificación formal con Lean 4 10

Documentación literaria para la auditoría 10

Evaluación y resultados 10

Capítulo 6 — ConclusionesCapítulo 7 — Trabajos finales

Referencias bibliográficas 13

Capítulo 1 — Introducción

Planteamiento del problema

El desarrollo de software crítico, como el utilizado en el protocolo Triage de Manchester —un sistema de clasificación de pacientes basado en prioridades clínicas— exige altos estándares de confiabilidad, seguridad y auditabilidad (*Emergency Triage*, 2014; Sorokin et al., 2024). Los Modelos de Lenguaje Avanzados (como en español se los llama a los Large Language Models, LLMs por sus siglas en inglés) han demostrado ser herramientas efectivas para la generación automática de código, mejorando la productividad y acelerando el ciclo de desarrollo (Kumar et al., 2025). Sin embargo, su adopción en sistemas críticos se ve limitada por tres problemas fundamentales.

Falta de determinismo y verificabilidad

Los LLMs generan código con alucinaciones (resultados plausibles pero incorrectos) y comportamientos no deterministas, lo que compromete la fiabilidad en entornos donde la precisión es crítica (Koomullil, 2026; Mohsin et al., 2024). Los enfoques de traducción de requisitos en lenguaje natural a especificaciones formales (Ma et al., 2025) permiten atacar este problema desde la fase de especificación.

Auditoría humana insostenible

La revisión manual de código generado por LLMs es costosa y propensa a errores, especialmente en sistemas con requisitos complejos como el Triage de Manchester (Yuan et al., 2026), donde la ambigüedad en la interpretación de reglas clínicas (*Emergency Triage*, 2014) puede llevar a fallos graves.

Falta de integración sistemática

Aunque existen metodologías, técnicas y herramientas para reducir errores en el desarrollo tradicional de software, no hay una integración sistemática entre estas facilidades y los LLMs (Sorokin et al., 2024).

Preguntas e hipótesis

Pregunta principal

¿Cómo puede mejorarse, en una implementación del Triage de Manchester, la eficacia de los LLMs en el ciclo de vida de desarrollo mediante la integración de verificación formal y programación literaria?

Preguntas secundarias

- ¿Qué impacto tiene, durante el ciclo de desarrollo del protocolo Triage de Manchester, la integración de código generado por LLMs y la programación literaria en la legibilidad de la documentación interseccionada con el código y la eficiencia de la auditoría humana?
- ¿Cómo se diseñaría un sistema que, aplicado al protocolo Triage de Manchester, combine generación de código por LLMs, verificación formal, programación literaria y auditoría humana eficiente y eficaz?

Hipótesis

La integración de verificación formal y programación literaria en el ciclo de desarrollo asistido por LLMs incrementa la corrección del código generado y reduce el esfuerzo de auditoría humana, sin sacrificar la velocidad de desarrollo, incluso en dominios de alta criticidad como el triage hospitalario.

Objetivos

Objetivo general

Proponer e implementar un marco neuro-simbólico que integre generación de código por LLMs, verificación formal y programación literaria para el desarrollo de software crítico, tomando como caso de estudio el protocolo Triage de Manchester.

Objetivos específicos

- Analizar la viabilidad de un sistema neuro-simbólico que permita a los LLMs generar código verificado formalmente, con documentación clara (programación literaria) y retroalimentación automatizada (Yuan et al., 2026).
- Diseñar un flujo de trabajo que integre generación de código por LLMs a partir de requisitos clínicos (ej.: «Si el paciente tiene dolor torácico y presión arterial < 90, asignar prioridad 1»), verificación formal del código generado y generación de documentación legible para auditores humanos (médicos o ingenieros) mediante programación literaria.
- Implementar un prototipo aplicado al Triage de Manchester, que traduzca requisitos clínicos a código verificable y genere documentación auditable.
- Evaluar el prototipo con técnicas cuantitativas (bugs, cumplimiento de invariantes) y cualitativas (usabilidad de la documentación para auditores).

Justificación

Esta investigación contribuye al campo de la ingeniería de software y la inteligencia artificial al proponer un marco que integre LLMs, verificación formal y programación literaria. La verificación formal permite asegurar que el código cumple con propiedades críticas (Moura & Ullrich, 2021), mientras que la programación literaria mejora la claridad y estructura del código, facilitando la auditoría humana (Yuan et al., 2026; Zhang et al., 2024). Este enfoque neuro-simbólico, combinación de aprendizaje automático y razonamiento lógico, busca generar código no solo sintácticamente correcto, sino semánticamente válido y verificable (Ma et al., 2025).

Marco de referencia

Programación literaria

La programación literaria, concebida por Knuth (Knuth, 1984), intercala narrativa explicativa y código ejecutable en un mismo documento fuente, orientado primordialmente a lectores humanos. Trabajos recientes retoman esta idea en la era de los LLMs para mejorar la generación de código en proyectos de gran escala (Zhang et al., 2024), y estudian su rol como interfaz de validación y trazabilidad para no programadores y auditores (Candela et al., 2023; Yuan et al., 2026).

Verificación formal y asistentes de pruebas

Los asistentes de pruebas como Lean 4 permiten verificar formalmente que un programa cumple con propiedades especificadas (Moura & Ullrich, 2021). El ecosistema actual incluye herramientas como LeanDojo, que provee datos y un entorno programable para trabajar con Lean en contextos de aprendizaje automático (Yang et al., 2023).

Sistemas de triage y software clínico

El Manchester Triage System se sustenta en flujogramas y discriminadores (*Emergency Triage*, 2014) que han sido estudiados en relación con la construcción de software clínico seguro (Zlotnik et al., 2016), con estudios de concordancia entre pacientes y sistemas de autotriage (Önler et al., 2026) y de rendimiento comparativo entre LLMs y profesionales en triage (Masanneck et al., 2024).

Estado del arte

- **Generación de código con LLMs:** evaluaciones recientes miden el impacto real de la IA en la productividad de desarrolladores (Kumar et al., 2025) y proponen marcos para evaluar la confianza en el código generado (Mohsin et al., 2024).

- **Traducción de requisitos a especificaciones formales:** enfoques automáticos que usan descomposición semántica jerárquica con LLMs para traducir requisitos en lenguaje natural a lógica temporal (LTL) (Ma et al., 2025).
- **Verificación continua y aseguramiento:** herramientas de creación continua de casos de aseguramiento (assurance cases) para sistemas críticos (Sorokin et al., 2024).
- **Programación literaria y auditoría:** renovación de la programación literaria con LLMs (Zhang et al., 2024), secuencias GP3 para validar la adherencia de código a la narrativa (Balaskas, 2026), experiencias con estándares de documentación para auditores (Candela et al., 2023) y validación humana de código LLM mediante programación literaria verificable (Yuan et al., 2026).

Aspectos metodológicos

Paradigma

El paradigma elegido es el pragmatismo. Este paradigma permite combinar validación formal con Lean 4, documentación narrativa (literate programming) y feedback humano, evaluando cada componente por su contribución a la generación de software correcto. El conocimiento válido se define por su capacidad para resolver problemas concretos (Peirce, 1878), en este caso: reducir errores en el código generado por LLMs y mejorar la comprensión del código por parte de los desarrolladores.

Método

Usaremos un método mixto, juntando cualitativo y cuantitativo.

Técnicas cuantitativas

- Experimentos controlados para medir el número de bugs y el cumplimiento de invariantes en el código generado (Ma et al., 2025).
- Benchmarking con herramientas de verificación formal que provee el ecosistema de Lean 4 para evaluar la corrección del código (Moura & Ullrich, 2021).

Técnicas cualitativas

- Análisis de usabilidad de la documentación generada (mediante encuestas a auditores humanos).
- Entrevistas a desarrolladores para evaluar la eficiencia de la auditoría.

Capítulo 2 — Marco teórico

TBD.

LLMs y generación de código

Verificación formal y Lean 4

Programación literaria

Auditoría humana y trazabilidad

Capítulo 3 — Estado del arte

TBD.

Generación de código asistida por LLMs

Traducción de requisitos a especificaciones formales

Programación literaria y auditoría en la era de los LLMs

Capítulo 4 — Marco metodológico

TBD.

Paradigma y enfoque

Diseño de la investigación

Técnicas de recolección y análisis

Protocolo experimental

Capítulo 5 — Diseño e implementación del prototipo

TBD.

Arquitectura del sistema

Generación de código a partir de requisitos clínicos

Verificación formal con Lean 4

Documentación literaria para la auditoría

Evaluación y resultados

Capítulo 6 — Conclusiones

TBD.

Capítulo 7 — Trabajos finales

TBD.

Referencias bibliográficas

Balaskas, N. (2026). gp3sequences: An Auditable and Reproducible Framework for Ordered Categorical Sequence Analysis in R. *AppliedMath*, *6*(10), 169. https://doi.org/10.3390/appliedmath6100169

Baudart, G., Lelarge, M., Stérin, T., & Viennot, J. (2026). *Putnam 2025 Problems in Rocq using Opus 4.6 and Rocq-MCP*. https://arxiv.org/abs/2603.20405

Candela, G., Sanyal, T., Qayyum, A., Luaces, M., & Laredo, J. (2023). An Approach to Assess the Quality of Jupyter Projects Published by Institutions. *Journal of the Association for Information Science and Technology*, *44*(4), 537-552. https://doi.org/10.1002/asi.24835

Emergency Triage. (2014). Wiley-Blackwell / BMJ Books. https://www.wiley.com/en-us/Emergency+Triage,+3rd+Edition-p-9781118299067

Ju, H., Gao, G., Jiang, J., Wu, B., Sun, Z., Liu, S., Chen, L., Wang, Y., Wang, Y., Wang, Z., He, W., Wu, P., Xiao, L., Liu, R., Dai, B., & Dong, B. (2026). *Automated Conjecture Resolution with Formal Verification*. https://arxiv.org/abs/2604.03789

Knuth, D. E. (1984). Literate Programming. *The Computer Journal*, *27*(2), 97-111. https://doi.org/10.1093/comjnl/27.2.97

Koomullil, G. (2026). *Proof-Carrying Certificates for LLM Pipelines: A Trust-Boundary Architecture*. https://arxiv.org/abs/2605.16407

Kumar, A., Khare, V., Sharma, D., Kumar, S., Saini, V., Yadav, A., Jain, S., Rana, A., Verma, P., Meena, V., & Edubilli, A. (2025). Intuition to Evidence: Measuring AI's True Impact on Developer Productivity. *ArXiv*. https://doi.org/10.48550/arXiv.2509.19708

Larrosa Martínez, A. (2026). *AuditFlow: plataforma en Python para automatizar procesos contables de auditoría* [Master's thesis]. https://riunet.upv.es/handle/10251/239444

Ma, Z., Wen, C., Su, Z., Liang, X., Tian, C., Qin, S., & Yang, M. (2025). Bridging Natural Language and Formal Specification-Automated Translation of Software Requirements to

LTL via Hierarchical Semantics Decomposition Using LLMs. *ArXiv*. https://doi.org/10.48550/arXiv.2512.17334

Masanneck, L., Schmidt, L., Seifert, A., Kölsche, T., Huntemann, N., Jansen, R., Mehsin, M., Bernhard, M., Meuth, S. G., Böhm, L., & Pawlitzki, M. (2024). Triage Performance Across Large Language Models, ChatGPT, and Untrained Doctors in Emergency Medicine: Comparative Study. *Journal of Medical Internet Research*, *26*, e53297. https://doi.org/10.2196/53297

Mohsin, A., Janicke, H., Wood, A., Sarker, I. H., Maglaras, L., & Janjua, N. (2024). Can We Trust Large Language Models Generated Code? A Framework for In-Context Learning, Security Patterns, and Code Evaluations Across Diverse LLMs. *ArXiv*. https://doi.org/10.48550/arXiv.2406.12513

Moura, L. de, & Ullrich, S. (2021). The Lean 4 Theorem Prover and Programming Language. *Computer Aided Verification (CAV 2021)*, *12759*, 625-670. https://doi.org/10.1007/978-3--030-81685-8_24

Önler, A. B., Serin, S., & Caglar, B. (2026). Agreement and Classification Performance of a Mobile Self-Triage Application Based on the Manchester Triage System in Hemodynamically Stable Adult Walk-In Emergency Department Patients: A Prospective Observational Study. *Journal of Clinical Medicine*, *15*(17), 6702. https://doi.org/10.3390/jcm15176702

Peirce, C. S. (1878). How to make our ideas clear. *Popular Science Monthly*, *12*, 286-302.

Song, P., Yang, K., & Anandkumar, A. (2024). *Lean Copilot: Large Language Models as Copilots for Theorem Proving in Lean*. https://arxiv.org/abs/2404.12534

Sorokin, L., Bouchekir, R., Beyene, T. A., Liao, B., & Molin, A. (2024). Towards Continuous Assurance Case Creation for ADS with the Evidential Tool Bus. *ArXiv*. https://doi.org/10.48550/arXiv.2403.01918

Xin, J., Schneidman, A., Cummins, C., Ram, K., Ganesh, S., & Limperg, J. (2026). *AXLE: A Cloud Infrastructure for Lean 4 Theorem Proving Utilities*. https://arxiv.org/abs/2606.26442

Yang, K., Swope, A. M., Gu, A., Chalamala, R., Song, P., Yu, S., Godil, S., Prenger, R., & Anandkumar, A. (2023). *LeanDojo: Theorem Proving with Retrieval-Augmented Language Models*. https://arxiv.org/abs/2306.15626

Yuan, Z., Lu, W., Wu, H., Jin, D., & Wu, C. (2026). Guiding Human Validation of LLM-Generated Code via Verifiable Literate Programming. *ArXiv*. https://doi.org/10.48550/arXiv.2607.02333

Zhang, W., Li, Y., Dong, Z., Wu, Y., Zhou, Y., Wang, D., Xing, S., Zhou, C., & Shen, D. (2024). Renaissance of Literate Programming in the Era of LLMs: Enhancing LLM-Based Code Generation in Large-Scale Projects. *ArXiv*. https://doi.org/10.48550/arXiv.2502.17441

Zheng, K., Han, J. M., & Polu, S. (2021). *MiniF2F: a cross-system benchmark for formal Olympiad--level mathematics*. https://arxiv.org/abs/2109.00110

Zlotnik, A., Alfaro, M. C., Pérez, M. C. P., Gallardo-Antolín, A., & Martínez, J. M. M. (2016). Building a Decision Support System for Inpatient Admission Prediction With the Manchester Triage System and Administrative Check-in Variables. *CIN: Computers, Informatics, Nursing*, *34*(5), 224-230. https://doi.org/10.1097/CIN.0000000000000230